

Práctica final: Implementación de un RTC en Raspberry Pi

Yonathan López Mejía, Manuel Santiago Velásquez López
Departamento de Ingeniería Electrónica
Universidad de Antioquia
Medellín, Colombia

I. INTRODUCCIÓN

NO es extraño que los sistemas en tiempo real se desplieguen en entornos sin conexión a internet y, por tanto, no dispongan de protocolos de sincronización temporal como NTP. Esta ausencia puede ocasionar imprecisiones en tareas que requieren una referencia temporal confiable respecto al presente. De igual forma, una gran variedad de dispositivos —como microcontroladores y computadores de placa reducida— carecen de un reloj integrado con capacidad de mantener una referencia precisa del tiempo actual. En este sentido, los periféricos conocidos como Relojes de Tiempo Real (RTC) constituyen una solución inmediata para estos y otros escenarios similares.

En esta práctica se realizó una primera aproximación al protocolo NTP, se configuró una Raspberry Pi para utilizar como fuente de reloj un módulo RTC DS3231 y así mantener su marca temporal actualizada sin depender del acceso a internet. Finalmente, se instaló un kernel Linux con extensiones para tiempo real y, junto con el RTC, se ejecutaron y analizaron dos aplicaciones en C: una que realizaba la lectura del dispositivo mapeado en el sistema operativo y otra que implementaba una aplicación concurrente de tiempo real con tres tareas planificadas mediante Rate Monotonic Scheduling (RMS), utilizando como fuente de *tick* las interrupciones de temporización generadas por el kernel esto con el fin de observar el cumplimiento de la planificación de tareas sobre RMS.

II. OBJETIVOS

- Integrar y verificar un dispositivo tipo Reloj de Tiempo Real (RTC), modelo DS3231, con una Raspberry Pi.
- Configurar el funcionamiento del RTC mediante el sistema operativo Raspberry Pi OS.
- Validar su operación a través de diferentes códigos fuente en ANSI C usando el estándar POSIX.

III. PREGUNTAS ORIENTADORAS

¿Qué es el Network Time Protocol (NTP) y cuál es su rol en los dispositivos Raspberry Pi?

El *Network Time Protocol* (NTP) [1] es un protocolo de capa de aplicación que funciona sobre *User Datagram Protocol* (UDP), utilizando el puerto 123, y permite sincronizar la fecha y hora de dispositivos conectados a una red. Es el mecanismo empleado por sistemas operativos como UNIX, Linux y Windows para obtener y mantener una hora del sistema precisa.

Sus principales características son:

- Las redes NTP suelen obtener la información de fecha y hora desde una fuente autorizada, como un reloj atómico o un reloj de radio. No obstante, también es posible, aunque relativamente raro, implementar una red NTP completamente separada de Internet, en la que uno o varios dispositivos actúan como fuentes internas de tiempo.
- NTP es altamente eficiente: en condiciones normales no necesita más de un paquete por minuto para mantener la sincronización, lo que permite una precisión del orden de 1 milisegundo entre máquinas.
- El protocolo utiliza un sistema jerárquico de estratos (*stratum*) para indicar la distancia entre una máquina y la fuente original de tiempo. El *stratum 0* corresponde a los relojes de alta precisión; el *stratum 1* a los dispositivos que se sincronizan directamente con ellos; el *stratum 2* a quienes se sincronizan con los de *stratum 1*; y así sucesivamente hasta *stratum 15*. El *stratum 16* identifica a los dispositivos que no están sincronizados. La Figura 1 ilustra esta estructura. Los dispositivos de un mismo estrato también pueden intercambiar información NTP entre sí, mediante un proceso conocido como *symmetric peering*, con el fin de mejorar la confiabilidad del reloj.
- NTP incorpora mecanismos para prevenir sincronizaciones erróneas. Un dispositivo nunca se sincroniza con otro que no esté sincronizado, y compara la información de múltiples máquinas para descartar aquellas cuyas mediciones difieran significativamente del resto, incluso si estas pertenecen a un estrato inferior.

La importancia de NTP para una Raspberry Pi radica en que este dispositivo no incorpora un reloj en tiempo real (RTC), por lo que pierde su configuración de fecha y hora cada vez que se apaga. Para restablecer la hora correctamente al encender, necesita de NTP. Una sincronización incorrecta puede causar desde inconvenientes relativamente simples, como fallos en la verificación de certificados o problemas de conexión a internet, hasta riesgos severos en sistemas críticos que requieren alta precisión temporal.

¿Qué sucede con la hora de un sistema basado en Raspberry Pi si este se desconecta de la alimentación por 10 minutos sin conexión a NTP?

Para comprender este comportamiento es necesario conocer el concepto de *fake-hwclock* [2]. Muchos sistemas Linux utilizados en dispositivos embebidos —incluida la Raspberry Pi— no cuentan con un reloj de hardware real (RTC) o no pueden acceder a él durante la secuencia de arranque. Como consecuencia, después de un reinicio la fecha y hora del sistema quedan atrasadas.

Una solución básica sería regresar la fecha a un punto conocido como el Epoch, pero si el sistema regresara simplemente esta fecha, podrían presentarse problemas: algunos servicios fallan al detectar un retroceso brusco en el tiempo y, además, no siempre es posible conectarse inmediatamente a un servidor NTP para corregir la hora.

Para mitigar esto, Linux incluye el programa *fake-hwclock*. Su función es:

- Guardar la última marca de tiempo correcta durante el apagado del sistema.
- Restaurar dicha marca de tiempo lo antes posible durante el arranque.
- Cuando se integra con un *daemon* que ejecute programas de manera periódica puede guardar la última marca correcta cada determinado tiempo.

De esta manera, *fake-hwclock* simula un reloj de hardware empleando únicamente software. Sin embargo, es importante tener en cuenta que esta herramienta **no lleva la cuenta del tiempo mientras el sistema está apagado** por ello, la última hora disponible siempre estará atrasada el tiempo que la máquina haya estado apagada.

En el escenario planteado, cuando la Raspberry Pi vuelva a encender sin acceso a NTP, la hora del sistema estará atrasada 10 minutos respecto al instante actual.

Debe notarse que el objetivo de esta herramienta es únicamente proporcionar una referencia temporal razonable hasta que un RTC externo o un servicio NTP puedan actualizar la hora a un valor correcto.

¿Por qué es importante desactivar el fake-hwclock en presencia de un RTC real?

El problema principal es que *fake-hwclock* ejecuta un *script* durante el arranque del sistema que sobrescribe la hora del sistema con un valor almacenado previamente en la tarjeta SD. Esto ocurre incluso cuando existe un RTC totalmente funcional como el DS3231. Por tanto, aunque el RTC mantenga una medición precisa del tiempo, el sistema acaba utilizando los valores desactualizados proporcionados por *fake-hwclock*.

Por esta razón es necesario desactivar *fake-hwclock* para permitir que el sistema use, sin interferencias, la marca de tiempo proveniente del RTC. No obstante, esta solución implica un riesgo: si la batería del RTC se agota y la Raspberry Pi se reinicia, la hora que restablezca el sistema podría ser tan antigua que cause fallos inesperados, tal como menciona el usuario *sveteb* en [3].

Una alternativa propuesta por dicha persona consiste en conservar *fake-hwclock* como mecanismo de respaldo y crear un *script* que, durante el arranque, fuerce al sistema a sincronizarse con el RTC siempre que éste esté disponible. Sin embargo, esta solución excede el alcance de la práctica actual y se propone como mejora para versiones futuras.

¿Qué diferencias existen entre los chips DS1307 y DS3231?

La Tabla I resume las diferencias más importantes entre ambos modelos de RTC. Los criterios principales de comparación son: el rango de alimentación, el consumo energético en modo

de espera y activo, la precisión del reloj, las características del bus I²C y las capacidades adicionales que aportan sus registros internos. Bajo estos parámetros, el DS3231 se considera superior al DS1307: ofrece un rango de voltaje compatible con sistemas embebidos modernos, menor deriva temporal debido a su oscilador interno compensado por temperatura, consumo reducido en modo activo aunque la corriente usada para la compensación de temperatura es relativamente alta (575 μ A a 3.3V) y funcionalidades avanzadas como el uso de alarmas basadas en distintos tipos de coincidencias de fecha y hora. Por otro lado, el DS1307 es un dispositivo interesante por un motivo ajeno a su función de RTC y es por la presencia de 56 bytes de RAM que pueden escribirse y leerse por I²C, esto podría ser útil en dispositivos con baja RAM para almacenar temporales de poco uso. Algunas versiones del DS3231 como el usado por el equipo en esta práctica tienen una funcionalidad de almacenamiento persistente tipo EEPROM, pero esta no es una capacidad del chip en sí sino del circuito donde se monta y que aprovecha el I²C para realizar escrituras y lecturas de datos persistentes.

¿Qué beneficios aporta un módulo RTC en un sistema embebido sin acceso a red?

En muchos sistemas embebidos desplegados en entornos remotos no existe acceso a internet, pero aun así deben ejecutar tareas sensibles al tiempo, como el monitoreo periódico de variables en ecosistemas remotos, la activación de instrumentos en momentos específicos o el envío de datos periódicos a través de protocolos de comunicación que no dependen de la red. Además, estos sistemas pueden requerir reinicios ocasionales para aplicar actualizaciones o recuperarse de fallos.

En este contexto, es fundamental mantener una referencia temporal lo más cercana posible al tiempo real. Un módulo RTC proporciona precisamente esta capacidad: ofrece una fuente estable y persistente de tiempo incluso cuando el sistema se encuentra apagado o aislado, o sirve como respaldo cuando no existen otros mecanismos de sincronización temporal.

Gracias a ello, los RTC permiten que dispositivos autónomos continúen cumpliendo sus funciones de manera efectiva, siempre que la deriva temporal del RTC sea adecuada para la aplicación. No obstante, los modelos analizados aquí no deben emplearse en sistemas críticos, ya que su precisión sigue siendo insuficiente para aplicaciones con requisitos temporales estrictos.

IV. CONFIGURACIÓN DEL DS3231 EN LA RASPBERRY PI

Conexión del módulo RTC DS3231 a la Raspberry Pi

Se utilizó una Raspberry Pi Modelo 3B+ en lugar del modelo 4. No existen diferencias significativas en lo referente a este laboratorio. El montaje es el mostrado en la figura 2.

Habilitación de la interfaz I2C en el Raspberry Pi OS

El archivo `/boot/config.txt` debía editarse en versiones antiguas del Raspberry Pi OS, en la versión que se utiliza en este laboratorio y basada en Debian Estable Trixie dicho archivo está en `/boot/firmware/config.txt`. Las figuras 3 y 4 muestran la configuración pedida.

TABLE I
COMPARACIÓN ENTRE LOS RTC DS1307 Y DS3231

Característica	DS1307 [4]	DS3231 [5]
Alimentación (Típica)	5V	3.3V
Registros	8B	19B
SRAM	(8Bx56)	0
Alarmas (Int)	0	2
Reloj	Cristal externo	Cristal interno
Compensación temp.	No	Sí (TXCO)
Corriente de compensación de temperatura	0	575 μ A
Precisión	Depende del oscilador	± 2 ppm (0°C to +40°C) ± 3.5 ppm (-40°C to +85°C)
SQW	1Hz, 4KHz, 8KHz, 32KHz	1Hz, 4KHz, 8KHz. Tiene un pin dedicado para los 32KHz
I2C Max Freq	100kHz	400kHz
Corriente en modo activo	1 mA	200 μ A, VCC=3.3
Corriente standby	200 μ A	110 μ A, VCC=3.3

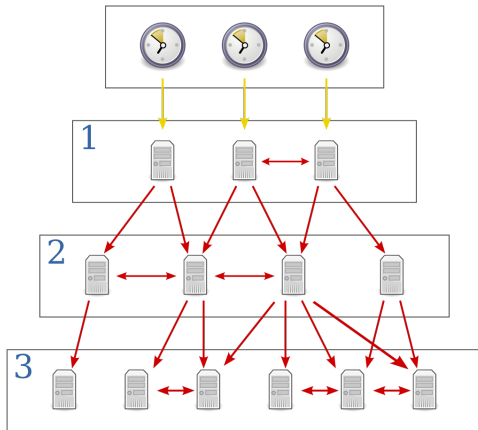


Fig. 1. Estratificación en NTP. Tomado desde Wikipedia

Comprobar la detección del RTC DS3231 en el bus I2C

El nombre del paquete en Debian Trixie es `i2c-tools` en lugar de `I2C-tools` y como `apt` es sensible a mayúsculas genera el error que se observa en la figura 5. Finalmente, se comprueba en la figura 6 la detección del RTC con dirección 0x68 y del módulo externo EEPROM del dispositivo que monta el DS3231 que es la 0x57. Nótese que nuevamente el comando difiere al de la guía en las mayúsculas.

Configurar el sistema operativo para usar el RTC DS3231

Para que el sistema operativo reconozca un RTC externo conectado mediante I2C, es necesario modificar el *Device Tree* utilizado por el kernel. Esto se realiza mediante la variable `dtoverlay` (Figura 7), a la cual se le asigna un *overlay* que describe tanto el tipo de dispositivo (`i2c-rtc`) como el modelo particular (`ds3231`). De esta manera es posible añadir la definición del hardware y cargar el controlador correspondiente sin necesidad de modificar ni recompilar el

kernel [6]. Como nota adicional, el RTOS ZephyrOS utiliza este mismo mecanismo para abstraer los controladores de los periféricos de la implementación de la funcionalidad en el código.

Finalmente, en la figura 8 se ilustra el resultado satisfactorio de la configuración del periférico. La utilidad *diagnostic messages* muestra el registro por parte del sistema operativo del RTC y la primera lectura de fecha y hora del dispositivo. Por otro lado, el `i2cdetect` en lugar de mostrar la dirección del dispositivo muestra las letras UU indicando dicho periférico es ahora gestionado por el kernel.

Deshabilitación del fake-hwclock y habilitación del uso del RTC real

La figura 9 muestra que efectivamente el *fake-hwclock* ha sido eliminado, por otro lado, se propone una solución más limpia para evitar la sincronización con un servidor NTP y es simplemente desactivar el *daemon* de la sincronización. Esto se logra, en `systemd` con dos comandos: `sudo systemctl stop systemd-timesyncd` y `sudo systemctl disable systemd-timesyncd`, el primero para el *daemon* si se está ejecutando y el segundo lo desactiva para que no se reactive en el reinicio. Esto es suficiente para permitir que el RTC sea la fuente primaria de tiempo.

Por otro lado, la figura 10 muestra el resultado de ejecutar el comando `timedatectl`. Los campos **NTP service** y **System clock synchronized** aparecen como deshabilitados indicando que no hay ningún servicio de sincronización de fecha y hora desde internet activo. Finalmente, la figura 11 muestra cómo sería la salida para un sistema donde la sincronización esté activada.

Sincronización de la hora y verificación del funcionamiento del RTC

Para validar la sincronización del RTC, se utilizó como referencia la hora oficial proporcionada por el Instituto Nacional de Metrología [7]. Primero, el tiempo del RTC se ajustó tomando como base dicho sitio web y, después, se realizó la primera verificación: la hora legal colombiana en el instante de la medición se muestra en la Fig. 12, mientras que la lectura obtenida del RTC se presenta en la Fig. 13. En esta comparación inicial se observa una coincidencia prácticamente exacta entre ambos valores.

Posteriormente, la Raspberry Pi se mantuvo completamente apagada y desconectada de la energía durante aproximadamente 45 minutos y se repitió el procedimiento de verificación. Los resultados se ilustran en las Fig. 14 y 15. Es importante destacar que, en esta última, la utilidad `dmesg` muestra el registro del RTC al momento del arranque mientras que `timedatectl` muestra la hora del sistema y el estado del servicio NTP, en el primer caso hay un desfase de 20 minutos entre el momento en que se enciende el sistema y se realiza la nueva prueba, y en el segundo caso hay un desfase 13 segundos entre el momento en que se lee la hora del RTC y luego se escribe el comando en consola. Todos estos resultados confirman que se realizó el procedimiento de forma exitosa.

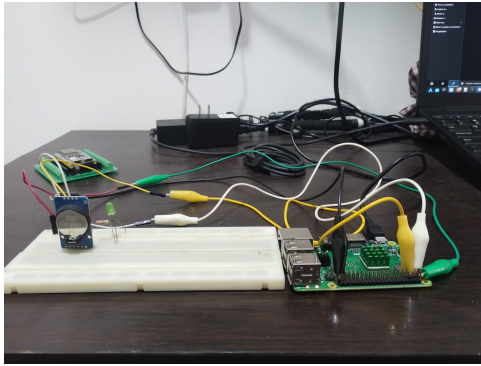


Fig. 2. Montaje DS3231 en Raspberry Pi Modelo 3B+

```
GNU nano 8.4 /boot/firmware/config.txt
# For more options and information see
# http://rptl.io/config.txt
# Some settings may impact device functionality. See link above for details
# Uncomment some or all of these to enable the optional hardware interfaces
dtparam=i2c_arm=on
dtparam=i2s=on
dtparam=spi=on
# Enable audio (loads snd_bcm2835)
dtparam=audio=on
```

Fig. 3. Activación de los controladores del kernel para I2C en el arranque

```
yonathan@raspi:~$ dmesg | grep i2c
[ 7.700040] i2c_dev: i2c /dev entries driver
```

Fig. 4. Verificación de la activación del controlador I2C

```
yonathan@raspi:~$ sudo apt install i2c-tools
Error: No se ha podido localizar el paquete i2c-tools
yonathan@raspi:~$ sudo apt install i2c-tools
i2c-tools ya está en su versión más reciente (4.4-2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
yonathan@raspi:~$
```

Fig. 5. Instalación de I2C tools y mensaje de error

```
yonathan@raspi:~$ sudo i2cdetect -y 1
sudo: i2cdetect: command not found
yonathan@raspi:~$ sudo i2cdetect -y 1
0 1 2 3 4 5 6 7 8 9 a b c d e f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- 57 -- -- -- -- -- -- --
60: -- -- -- -- -- -- 68 -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

Fig. 6. Detección del DS3231 y de la memoria EEPROM en el bus I2C

```
GNU nano 8.4 /boot/firmware/config.txt
# For more options and information see
# http://rptl.io/config.txt
# Some settings may impact device functionality. See link above for details
# Uncomment some or all of these to enable the optional hardware interfaces
dtparam=i2c_arm=on
dtoverlay=i2c-rtc,ds3231
```

Fig. 7. Modificación del device tree para cargar el driver del DS3231

```
yonathan@raspi:~$ sudo i2cdetect -y 1
0 1 2 3 4 5 6 7 8 9 a b c d e f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- 57 -- -- -- -- -- -- --
60: -- -- -- -- -- -- 68 -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- --

yonathan@raspi:~$ sudo dmesg | grep rtc
3.183380] rtc-ds1307 1-0068: SET TIME!
3.185390] rtc-ds1307 1-0068: registered as rtc0
3.188444] rtc-ds1307 1-0068: setting system clock to 2025-11-16T20:12:02 UTC (1763323922)
3.364450] vc4-drm soc:gpu: bound 3f286000.pixelvalve (ops vc4_crtc_ops [vc4])
3.364745] vc4-drm soc:gpu: bound 3f287000.pixelvalve (ops vc4_crtc_ops [vc4])
3.365897] vc4-drm soc:gpu: bound 3f807000.pixelvalve (ops vc4_crtc_ops [vc4])
3.372214] vc4-drm soc:gpu: [drm] Cannot find any crtc or sizes
3.373727] vc4-drm soc:gpu: [drm] Cannot find any crtc or sizes
```

Fig. 8. Reconocimiento por parte del Kernel del RTC

```
yonathan@raspi:~$ sudo apt remove -y fake-hwclock
El paquete «fake-hwclock» no está instalado, no se eliminará
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
yonathan@raspi:~$ sudo update-rc.d -f fake-hwclock remove
```

Fig. 9. Confirmación de eliminación del fake hardware clock

```
yonathan@raspi:~$ timedatectl
Local time: dom 2025-11-16 15:40:55 -05
Universal time: dom 2025-11-16 20:40:55 UTC
RTC time: dom 2025-11-16 20:40:55
Time zone: America/Bogota (-05, -0500)
System clock synchronized: no
NTP service: inactive
RTC in local TZ: no
```

Fig. 10. Comando timedatectl con servicio NTP desactivado

```
yonathan@celesti ~]$ timedatectl
Local time: dom 2025-11-16 15:47:24 -05
Universal time: dom 2025-11-16 20:47:24 UTC
RTC time: dom 2025-11-16 20:47:24
Time zone: America/Bogota (-05, -0500)
System clock synchronized: yes
NTP service: active
RTC in local TZ: no
```

Fig. 11. Comando timedatectl con servicio NTP activado

HORA LEGAL
REPÚBLICA DE COLOMBIA

15:58:56

hora : minutos : segundos

(UTC - 5) = TIEMPO UNIVERSAL COORDINADO DISMINUIDO 5 HORAS

FECHA:

AÑO MES DÍA

2025-11-16

Fig. 12. Prueba 1: hora legal Colombiana

```
yonathan@raspi:~$ sudo hwclock -r
2025-11-16 15:58:56.324947-05:00
```

Fig. 13. Prueba 1: lectura del periférico

HORA LEGAL
REPÚBLICA DE COLOMBIA

17:03:03

hora : minutos : segundos

(UTC - 5) = TIEMPO UNIVERSAL COORDINADO DISMINUIDO 5 HORAS

Fig. 14. Prueba 2: hora legal Colombiana

```
yonathan@raspi:~$ sudo hwclock -r
2025-11-16 17:03:03.498065-05:00
yonathan@raspi:~$ timedatectl
Local time: dom 2025-11-16 17:03:16 -05
Universal time: dom 2025-11-16 22:03:16 UTC
RTC time: dom 2025-11-16 22:03:16
Time zone: America/Bogota (-05, -0500)
System clock synchronized: no
NTP service: inactive
RTC in local TZ: no
yonathan@raspi:~$ dmesg | grep rtc
2.546331] rtc-ds1307 1-0068: SET TIME!
2.548439] rtc-ds1307 1-0068: registered as rtc0
2.551357] rtc-ds1307 1-0068: setting system clock to 2025-11-16T17:03:12 UTC (1763329332)
2.712790] vc4-drm soc:gpu: bound 3f286000.pixelvalve (ops vc4_crtc_ops [vc4])
2.712890] vc4-drm soc:gpu: bound 3f287000.pixelvalve (ops vc4_crtc_ops [vc4])
2.713210] vc4-drm soc:gpu: bound 3f807000.pixelvalve (ops vc4_crtc_ops [vc4])
2.717010] vc4-drm soc:gpu: [drm] Cannot find any crtc or sizes
2.718324] vc4-drm soc:gpu: [drm] Cannot find any crtc or sizes
```

Fig. 15. Prueba 2: lectura del periférico

V. IMPLEMENTACIÓN DE CÓDIGO FUENTE DE TIEMPO REAL

Análisis del código de lectura del RTC

El programa desarrollado permite realizar una lectura directa por I2C del DS3231 cuando es controlado por el kernel de Linux mediante el archivo ubicado en el punto de montaje `/dev/rtc0`. Para esto, se incluyen los encabezados necesarios: `fcntl.h` para la función `open()` [8], `unistd.h` para

```

jonathan@raspi:~/tiempo_real/abi/LAB6_RTS/Prueba $ gcc -Wall testing_rtc.c -o testing_rtc
jonathan@raspi:~/tiempo_real/abi/LAB6_RTS/Prueba $ ./testing_rtc
Error abriendo el RTC: Permission denied
jonathan@raspi:~/tiempo_real/abi/LAB6_RTS/Prueba $ sudo ./testing_rtc

```

Fig. 16. Ejecución del primer programa

`close()` [9], `sys/ioctl.h` para el uso de `ioctl()` [10], y `linux/rtc.h`, que contiene las definiciones específicas de los drivers para RTC del kernel [11].

Inicialmente, se define la ruta del dispositivo: `/dev/rtc0`. Este archivo representa al RTC y, como todo en Linux y Unix es un archivo, este se abre con la llamada al sistema `open()` en modo de solo lectura porque solo se leerá la marca de tiempo. Esta función retorna un *file descriptor* (`fd`), que es un entero que identifica al archivo y con el se trabajará a continuación. Si la apertura falla, el programa informa el error llamando a `perror()` y finaliza con un código distinto de cero.

Luego, se declara una estructura del tipo `struct rtc_time`, la cual contiene los campos necesarios para almacenar una marca temporal completa: segundos, minutos, horas, día del mes, mes y año. Los campos día de la semana, del año y los indicadores de horario de verano o invierno, no se usan. [11]

La lectura del RTC se realiza con la función `ioctl()` usando el comando `RTC_RD_TIME`, que solicita al kernel obtener la hora actual del dispositivo y almacenarla en la estructura anterior [11]. Si la operación retorna un valor negativo, se reporta el error, se cierra el archivo y se termina la ejecución.

Después, la fecha y hora obtenidas se muestran en pantalla en formato `YYYY-MM-DD HH:MM:SS`. Debido a la forma en que la implementación POSIX representa algunos campos (`tm_year` como años desde 1900 y `tm_mon` como un valor entre 0 y 11), se realizan los ajustes necesarios para mostrar los valores en el formato usual, debe notarse que en los registros del RTC estos valores van del 0 al 99 y del 1 al 12 respectivamente [5], así que este cambio no lo impone el periférico sino la implementación del controlador en Linux. Para terminar, el descriptor del dispositivo es cerrado, finalizando el programa.

El resultado de ejecutar el programa se observa en la figura 16, en este sentido, se evidencia que cuando se hace la ejecución usual `./testing_rtc` ocurre un error de permisos, esto sucede porque el programa trata de acceder a un dispositivo manejado por el kernel y aunque no accede directamente ya que se están usando llamadas al sistema, esta operación solo puede realizarla el usuario `root`, así que la ejecución correcta debe ser: `sudo ./testing_rtc`. Cuando la ejecución se realiza la salida tiene el formato exacto que se mencionó anteriormente.

INSTALACIÓN DE KERNEL EN TIEMPO REAL Y BENCHMARK

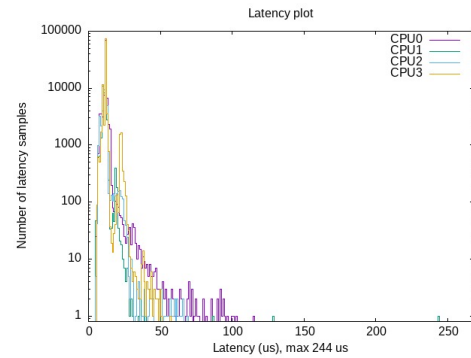
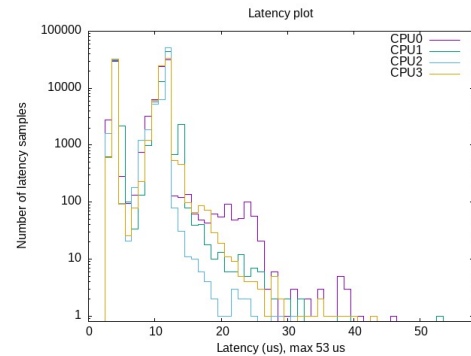
Antes de continuar con el desarrollo de la aplicación multi-hilo, se instaló y probó en el salón de clase un kernel de tiempo real obtenido desde los repositorios oficiales de Raspberry Pi OS, siguiendo lo indicado en este foro [3]. La figura 17 muestra el resultado del comando `uname -r` ejecutado en la

```

jonathan@raspi:~$ uname -r
6.12.47+rpt-rpi-v8-rt

```

Fig. 17. Ejecución del primer programa

Fig. 18. Resultado del *benchmark* para la versión *stock* del kernel 6.12Fig. 19. Resultado del *benchmark* para la versión RT del kernel

Raspberry Pi 3B+, lo cual confirma la correcta instalación y uso del kernel en tiempo real. Este será el núcleo empleado para la ejecución de la aplicación de tiempo real.

Posteriormente, se realizó una la misma prueba de estrés utilizando la utilidad `rt-tests` tanto con el kernel *stock* como con el kernel RT y cuyos resultados se presentan en las figuras 18 y 19 respectivamente. La primera evidencia importante es una reducción cercana a cinco veces en la latencia máxima al emplear el kernel RT. Además, se observa una distribución de latencias mucho más compacta a lo largo de las distintas ejecuciones, lo que demuestra la utilidad de este tipo de núcleos para aplicaciones que requieren baja latencia y bajo *jitter*.

Sin embargo, esto no implica que un kernel RT deba preferirse siempre sobre uno de propósito general. Tal como se observó en múltiples ocasiones, el kernel de tiempo real puede generar inestabilidad general del sistema cuando se ejecutan aplicaciones de uso general, como navegadores web. De esta forma, su uso debe limitarse a escenarios donde se requieran baja latencia, bajo *jitter* y alto determinismo.

APLICACIÓN MULTITHREAD DE TIEMPO-REAL Y CON PLANEACIÓN RMS

El código es una aplicación sencilla para validar el correcto funcionamiento del planificador bajo una política de RMS cuando se utiliza como fuente de reloj el RTC DS3231 el cual permite generar interrupciones a una frecuencia de 64Hz. Corre sobre una distribución Raspberry Pi OS basada en Debian Trixie y que utiliza la compilación del kernel RT oficial del proyecto Rapsberry. Las tareas que se planifican sobre RMS se observan en la tabla II. Existe una cuarta tarea asociada al hilo `rtc_thread` el cual funciona con la política por defecto del sistema operativo.

Análisis del código propuesto

Las funciones asociadas al código son las siguientes:

- **rtc_periodic_thread:** Es la función de entrada del hilo `rtc_thread`. En este hilo se realiza la configuración del RTC DS3231 mediante el controlador del kernel. Para esto se utiliza la función `ioctl()` [10] junto con los macros `RTC_IRQP_SET` y `RTC_PIE_ON`. El primero permite establecer la frecuencia (64 Hz) de las interrupciones periódicas (*ticks*), mientras que el segundo habilita dichas interrupciones [11]. Es importante indicar que estas interrupciones no vienen de las salidas físicas del DS3231, sino que son generadas internamente por los temporizadores del kernel. Por otro lado, el cuerpo de la función bloquea la ejecución del hilo mediante una llamada a `read()` sobre el descriptor asociado al RTC. Esta llamada se bloquea hasta que ocurre una interrupción, momento en el cual se accede de forma segura a la variable `tick_count` utilizando un `tick_mutex` y se incrementa. Cada vez que esto sucede significa que se ha generado un nuevo *tick*, por lo que se procede a notificar a las tareas periódicas para que verifiquen si deben ejecutarse en dicho instante usando `pthread_cond_broadcast()`. Posteriormente, el `mutex` se libera para permitir su uso por otras tareas. Este comportamiento se repite hasta que la variable `running` cambia su estado a bajo. Cuando esto ocurre, el hilo finaliza deshabilitando las interrupciones periódicas y cerrando el descriptor de archivo.
- **periodic_task():** Es la función de entrada para los tres hilos que implementan las tareas periódicas. Cada hilo inicializa la variable `next_release`, que representa el número de *ticks* correspondiente al siguiente instante en el que la tarea puede ejecutar su cuerpo principal. Para esto, se accede de forma serializada a la variable compartida `tick_count` mediante el `tick_mutex`. El cuerpo del `loop` principal consiste en intentar tomar el `tick_mutex`; una vez obtenido, la tarea valida si ya es momento de ejecutarse. Si aún no ha llegado su *release time*, el hilo se bloquea usando `pthread_cond_wait()`, esperando un nuevo *tick*. Cuando el hilo `rtc_thread` notifica a todas las tareas bloqueadas mediante la misma variable condicional,

la tarea de mayor prioridad que se encontraba esperando se despierta y verifica nuevamente el predicado (`tick_count < next_release`). Si finalmente ha llegado su instante de ejecución, la tarea actualiza su `next_release`, ejecuta su cuerpo y vuelve a esperar hasta su próxima activación, permitiendo que otras tareas de menor prioridad puedan ejecutarse posteriormente. Esto se repite, al igual que en el caso anterior, hasta que la variable `running` no cambie a estado bajo.

- **main():** En esta función se realiza la inicialización completa del sistema. Para garantizar un comportamiento determinista, lo primero que se hace es llamar a `mlockall()`, evitando fallos de página y bloqueando en memoria todas las páginas presentes y futuras del proceso, esto permite evitar penalizaciones al ocurrir fallos de paginación que aumentan el tiempo de ejecución de manera inesperada.

Luego, se define la frecuencia del reloj (64 Hz) y se inicializa un arreglo con los datos de las tareas periódicas, donde cada tarea contiene su identificador, su período en *ticks*, su prioridad asignada mediante asignación RMS y un nombre descriptivo.

Después, se crea el hilo `rtc_thread`, que genera los *ticks* utilizados como base temporal del sistema. A continuación, se configura la planificación de los tres hilos de las tareas periódicas. Cada uno se configura con política `SCHED_FIFO`, prioridades RMS y el parámetro `PTHREAD_EXPLICIT_SCHED` para asegurar que no se herede la configuración de planificación del hilo padre. Con lo anterior se crean los tres hilos usando `pthread_create()`.

El programa se ejecuta durante 20 segundos, al finalizar se establece la variable global `running` en cero, haciendo que todos los bucles de los hilos terminen su ejecución. Se realiza un `broadcast` sobre la variable condicional para despertar a cualquier hilo que permanezca bloqueado esperando un *tick*. Finalmente, el hilo principal espera la finalización de todos los hilos mediante `pthread_join()`, asegurando la finalización ordenada.

- **Exclusión mutua adicional:** Todas las tareas periódicas hacen uso de la llamada al sistema `printf()` como parte de su cuerpo principal, así que el *buffer* sobre el que se escribe funciona como un recurso compartido. En el código original esta llamada no está protegida por un `mutex` y esto puede causar condiciones de carrera, con lo cual, se solucionó dicho problema creando una nueva exclusión de nombre `printf_mutex`.

Resultados

Para la validación de los resultados se considera que, bajo un *scheduler* determinista con prioridades fijas y tareas estrictamente periódicas, si todas cumplen su *deadline* dentro de un hiperperíodo, entonces también lo harán en los hiperperíodos subsiguientes. En este caso dicha condición se cumple, puesto que el *kernel* utilizado está optimizado para aplicaciones de tiempo real y las tareas poseen prioridades fijas, como se explicó anteriormente.

TABLE II
TAREAS DE LA APLICACIÓN MULTITHILO

ID	Periodo (ticks)	Prioridad	Nombre
1	16	80	Tarea 1
2	32	70	Tarea 2
3	80	70	Tarea 2

El hiperperiodo, calculado en *ticks*, es el siguiente:

$$\text{Hiperperiodo} = \text{LCM}(16, 32, 80) = 160$$

En la Figura 20 se muestra el resultado de la ejecución de la aplicación para dos hiperperiodos consecutivos. La línea azul corresponde al primer hiperperiodo y la roja al segundo. Del análisis del primer hiperperiodo se pueden destacar los siguientes aspectos:

- La tarea 1 se ejecuta dos veces (en los *ticks* 16 y 32). Sin embargo, en el instante del segundo disparo también debe ejecutarse la tarea 2 (cuyo periodo es de 32 *ticks*). Como la tarea 2 tiene una prioridad inferior, la tarea 1 la releva y se ejecuta en ese *tick*, retrasando a la tarea 2 hasta el siguiente. Este comportamiento es el esperado y confirma el relevo de tareas de menor prioridad.
- Un resultado similar ocurre en el *tick* 80. Este instante es múltiplo tanto de 16 como de 80, por lo que coinciden los disparos de la tarea 1 (la de mayor prioridad) y de la tarea 3. Como es esperable, se ejecuta primero la tarea 1. La tarea 3 debería ejecutarse en el *tick* 81; sin embargo, su ejecución se retrasa hasta el *tick* 82 porque una tarea de mayor prioridad —bloqueada temporalmente mediante `usleep(50000)`— se despierta y releva a la tarea 3, posponiéndola un *tick* adicional.
- En el *tick* 160 coinciden nuevamente los disparos de las tres tareas, ya que dicho valor es múltiplo de 16, 32 y 80. La tarea 1 se ejecuta en ese instante, desplazando a las tareas 2 y 3. En el siguiente *tick*, la tarea 2 puede ejecutarse, lo que a su vez retrasa la ejecución de la tarea 3 un *tick* adicional. Este comportamiento confirma que la política de prioridades del planificador funciona correctamente.
- Una vez ocurre la ejecución correspondiente al *tick* 160, se completa el hiperperiodo. Desde este momento, las siguientes ejecuciones se repiten de manera idéntica al primer hiperperiodo, validándose así el comportamiento determinista esperado.
- El conteo correcto de los *ticks* y la impresión sin problema en la salida estándar demuestra que los recursos compartidos se están accediendo correctamente.

VI. CONCLUSIONES

- En sistemas de tiempo real la sincronización temporal es un asunto crítico, en muchos casos no es suficiente únicamente tener una fuente de sincronización como NTP o un RTC, por el contrario, deben existir alternativas que puedan cumplir con dicho trabajo de manera precisa incluso si alguna otra fuente deja de estar disponible de esta forma se obtiene un sistema con redundancia que

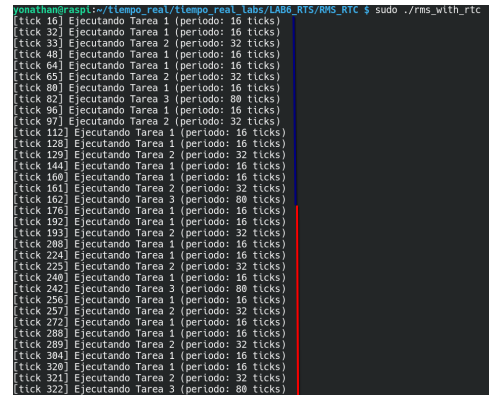


Fig. 20. Salida de la aplicación concurrente para dos hiperperiodos

puede continuar funcionando correctamente en caso de errores.

- En aplicaciones de tiempo real no solo es importante determinar correctamente la planificación de tareas y el uso de relojes de tiempo real, sino también que la infraestructura de software sobre la que se montan las aplicaciones tenga el soporte adecuado para tiempo real. En el caso de los sistemas que corren sobre Linux, la existencia de compilaciones de kernels RT oficiales y soportados que permiten tener planificación de tareas con relevos, prioridades y precisión para cumplir con los objetivos del sistema es una ventaja significativa. En esta práctica se mostró el funcionamiento de una aplicación sencilla con hilos y prioridades basadas en RMS que permitió demostrar la alta precisión que se puede alcanzar al utilizar núcleos dedicados al tiempo real.
- Si bien desactivar el *fake-hwclock* se mostró como una solución eficaz para garantizar que la sincronización del reloj del sistema con el DS3231 se hiciera de manera correcta, no debe olvidarse que dicho programa puede funcionar también como un elemento de redundancia permitiendo que, en caso de falla del RTC por daño o agotamiento de su batería, el sistema puede continuar funcionando con una marca temporal menos retrasada que la que tendría si el *fake-hwclock* estuviera desactivado. En este sentido, podrían explorarse alternativas como crear un script que sincronice el sistema a el *fake-hwclock* únicamente si no existe otra fuente de reloj de mayor precisión disponible.
- El RTC DS3231 es un dispositivo útil como fuente de temporización para la Raspberry Pi y cualquier dispositivo que no tenga un periférico interno dedicado a esta tarea en escenarios donde el acceso a internet esté limitado. Sin embargo, su todavía relativa baja precisión lo hace adecuado en aplicaciones de tiempo que real que no sean críticas, siendo necesario recurrir a opciones más robustas, y por tanto más caras, cuando se requiera utilizar RTCs en sistemas críticos.

REFERENCES

- [1] Cisco Systems, Inc., *Basic System Management Configuration Guide, Cisco IOS XE Release 3SE (Catalyst 3650 Switches)*,

- 3rd ed., Cisco, Internet, 2012, consultado: 2025-11-15. [Online]. Available: <https://www.cisco.com/c/dam/en/us/td/docs/ios-xml/ios/bsm/configuration/xr-3se/3650/bsm-xr-3se-3650-book.html>
- [2] Debian Manpages, *fake-hwclock (8) — Control fake hardware clock*, jessie ed., Debian Project, Internet, 2014, consultado: 2025-11-15. [Online]. Available: <https://manpages.debian.org/jessie/fake-hwclock/fake-hwclock.8.en.html>
- [3] Raspberry Pi Community, *Do I need to remove fake-hwclock if I use an RTC chip?*, n/a ed., Raspberry Pi Forums, Internet, 2021, consultado: 2025-11-15. [Online]. Available: <https://forums.raspberrypi.com/viewtopic.php?t=256726>
- [4] Analog Devices / Maxim Integrated, *DS1307 64×8, Serial, PC Real-Time Clock Data Sheet*, rev. 5 ed., Analog Devices, Inc., Internet, 2015, consultado: 2025-11-15. [Online]. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/ds1307.pdf>
- [5] —, *DS3231 Extremely Accurate PC-Integrated RTC/TCXO/Crystal Data Sheet*, rev. 10 ed., Analog Devices, Inc., Internet, 2015, consultado: 2025-11-15. [Online]. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/ds3231.pdf>
- [6] The Kernel Development Community, *Device Tree Overlay Notes*, v5.10 ed., Linux Kernel Documentation, Internet, 2020, consultado: 2025-11-16. [Online]. Available: <https://docs.kernel.org/devicetree/overlay-notes.html>
- [7] Instituto Nacional de Metrología (INM), *Hora Legal Colombiana*, n/a ed., INM República de Colombia, Internet, 2024, consultado: 2025-11-16. [Online]. Available: <https://horalegal.inm.gov.co/>
- [8] The Open Group, *fcntl.h — Header File for File Control Options*, issue 6, ieee std 1003.1-2004 ed., The Open Group, Internet, 2004, consultado: 2025-11-16. [Online]. Available: <https://pubs.opengroup.org/onlinepubs/000095399/basedefs/fcntl.h.html>
- [9] —, *unistd.h — Header File for Symbolic Constants, Types, and Miscellaneous Functions*, issue 7 ed., The Open Group, Internet, 2018, consultado: 2025-11-16. [Online]. Available: <https://pubs.opengroup.org/onlinepubs/7908799/xsh/unistd.h.html>
- [10] The Linux Programming Interface Community, *ioctl(2) — Control device*, man-pages 6.10 ed., man-pages (Linux Kernel and C Library), Internet, 2024, consultado: 2025-11-16. [Online]. Available: <https://man7.org/linux/man-pages/man2/ioctl.2.html>
- [11] —, *rtc(4) — real-time clock*, man-pages 6.10 ed., man-pages (Linux Kernel and C Library Documentation), Internet, 2024, consultado: 2025-11-16. [Online]. Available: <https://man7.org/linux/man-pages/man4/rtc.4.html>